Leveraging Human Insights for Enhanced LLM-based Code Repair

Yifan Zhang
Vanderbilt University
Nashville, USA

Kevin Leach
Vanderbilt University
Nashville, USA

Abstract

Large Language Models (LLMs) show promise for automating code repair but often lack the nuanced, iterative reasoning and effective use of historical context employed by human developers. We propose a framework to enhance LLM-based repair by incorporating human-inspired mechanisms: mining commit histories for recurring patterns, employing dynamic feedback loops, and facilitating reasoning over historical repair experiences. By extracting abstract fix patterns, iteratively refining patches using automated feedback (from tests and static analysis) alongside optional human guidance, and leveraging a vectorized repository of past experiences for context-aware reasoning, our approach seeks to improve automated patch generation. This framework aims to increase the accuracy and efficiency of repairs by guiding LLMs with more human-like, iterative, and context-grounded problem-solving strategies.

CCS Concepts

• **Software and its engineering** → **Software creation and management**; • **Security and privacy** → **Software and application security**.

Keywords

Automated Code Repair, Large Language Models, Software Vulnerabilities

ACM Reference Format:
Yifan Zhang and Kevin Leach. 2025. Leveraging Human Insights for Enhanced LLM-based Code Repair. In *33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion '25), June 23–28, 2025, Trondheim, Norway*. ACM, New York, NY, USA, 2 pages. https://doi.org/10.1145/3696630.3731672

1 Introduction

LLMs show promise in automated code repair, yet often fail to address underlying root causes or match human nuance, struggling with deep semantic understanding and the structural implications of code changes. Human developers effectively combine structural intuition, analogical reasoning from past experience, and iterative refinement based on concrete feedback. This work aims to bridge this gap by integrating mechanisms that emulate these human debugging processes—specifically structure-aware contextual reasoning and iterative feedback—into LLM-based repair.

This work is licensed under a Creative Commons Attribution 4.0 International License.
FSE Companion '25, Trondheim, Norway
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1276-0/2025/06
https://doi.org/10.1145/3696630.3731672

Motivated by these human strategies, we propose a system incorporating three core components: 1) mining commit histories for potentially reusable abstract fix patterns, 2) employing a dynamic feedback loop with automated checks (tests, static analysis) and optional structured developer guidance for systematic refinement, and 3) facilitating structure-guided reasoning over historical context. This reasoning leverages a repository of past fixes alongside detailed abstract interpretations (like Code Property Graphs - CPGs) of both current and historical code, enabling the analysis of structural analogies and the accurate adaptation of relevant patterns. This overall approach guides the LLM with processes mirroring human problem-solving to improve patch quality and efficiency.

Furthermore, the system enables continuous learning by assimilating validated repair strategies. By combining structure-guided historical reasoning with iterative feedback loops, we move beyond superficial pattern matching towards a more robust, context-aware repair process. The deep structural analysis informs the initial reasoning guided by history, while iterative feedback validates the resulting patches. This fosters a potent synergy for software maintenance, blending LLM scalability with structure-aware, human-like adaptation and learning.

2 Methodology

Our framework enhances LLM-based code repair by integrating multi-view code interpretations, iterative feedback, vectorized experience storage, and structure-guided reasoning over historical context. These components aim to provide the LLM with deeper program understanding and a more systematic, human-like debugging approach, leveraging abstract code structures to guide reasoning. Effective integration remains an active research challenge.

2.1 Multi-View Abstract Interpretation and Analysis

To comprehensively understand code for repair, we extract multiple graph representations capturing complementary structural and semantic properties: the Abstract Syntax Tree (G_{AST}), Control Flow Graph (G_{CFG}), and Program Dependence Graph (G_{PDG}). These are aggregated into a Code Property Graph (CPG):

$$G_{\text{CPG}}(c) = \alpha(G_{\text{AST}}(c), G_{\text{CFG}}(c), G_{\text{PDG}}(c)).$$

The CPG provides a rich, unified representation connecting syntax, control flow, and data dependencies. While its embedding $v_c = \phi(G_{\text{CPG}}(c))$ is computed primarily for efficient similarity assessment, the detailed CPG structure itself is crucial for subsequent analysis and reasoning steps, helping pinpoint bug-relevant elements. External findings (e.g., static analysis warnings) can also be mapped onto the CPG to highlight potential issues.

2.2 Iterative Refinement with Optional Human-in-the-Loop Feedback

Repair generation is often iterative. An initial patch p_0, generated via the structure-guided reasoning process (Sec 2.4), is evaluated using an automated function $f(p)$ assessing quality via tests and static analysis scores. Optional human input h_i (critiques, suggestions) can guide refinement at iteration i:

$$p_{i+1} = \text{LLM}\Big(\text{context}(c),\ p_i,\ f(p_i),\ h_i\Big).$$

The LLM interprets the combined feedback $f(p_i)$ and h_i to generate a revised patch p_{i+1}. The loop, using original context and current patch info, stops if $f(p_i) \geq \theta$ (and human needs met, if h_i given) or max iterations are reached. This allows progressive refinement based on concrete evidence and optional expert guidance.

2.3 Vectorized Experience Storage

Completed repair episodes are stored as vectors in a database $\mathcal{V}$. Each vector captures a problem-solution-outcome tuple:

$$v_{(c,p,f)} = \psi\Big(v_c,\ \text{repr}(p),\ \text{repr}(f(p)), \text{ref}(G_{\text{CPG}}(c))\Big).$$

Here, v_c is the CPG embedding for similarity matching, $\text{repr}(p)$ represents the patch, $\text{repr}(f(p))$ captures the outcome, and $\text{ref}(G_{\text{CPG}}(c))$ optionally stores a reference or key structural features of the original CPG for deeper analysis during reasoning. This database $\mathcal{V}$ forms a knowledge base linking code structures (via v_c and potentially $\text{ref}(G_{\text{CPG}})$) to repair strategies and outcomes.

2.4 Structure-Guided Reasoning with Historical Context

When encountering new buggy code c', we compute its CPG, $G_{\text{CPG}}(c')$, and embedding $v_{c'}$. We retrieve the k most similar experience vectors $\{v_j\}_{j=1}^{k}$ from $\mathcal{V}$ based on v_c similarity. Crucially, the reasoning leverages detailed CPG structures. A temporary state graph is constructed using key CPG elements and CPG-derived relationships (flows, dependencies, similarities) from both c' and relevant retrieved examples c_j.

The LLM performs structure-guided reasoning by navigating this state graph. It identifies the most promising reasoning path connecting problematic CPG patterns in c' to analogous structures in historical examples c_j linked to successful past patches p_j. Following this path, the LLM adapts the relevant historical transformation(s) and applies them to c' (locations identified via CPG) to synthesize the initial patch candidate p'_0. This process aims for more targeted and explainable reasoning.

3 Evaluation and Discussion

Our conceptual framework, planned for validation on benchmarks like **SVEN** [3] and **PrimeVul** [2], hypothesizes that integrating historical reasoning and iterative feedback improves patch quality and reduces iterations versus one-shot generation. Key metrics cover correctness (tests, static analysis), quality (complexity), and efficiency (iteration count). Evaluating optional human feedback (h_i) is challenging; approaches include simulation, user studies, or ablation of the automated loop ($f(p)$). The goal is to quantify the benefits of human-like iteration and context in automated repair.

4 Related Work

LLMs are increasingly applied to code repair and optimization [1, 4]. Relevant research explores enhancing LLMs via historical data reasoning or retrieval [5], iterative refinement loops [9], and their self-correction limits [8]. Work incorporating human cognitive signals [6, 11, 12] informs our optional human feedback design. Foundational code representation techniques, such as graph embeddings and contrastive learning [7, 10], underpin our multi-view interpretation and experience analysis components.

5 Conclusion and Future Work

Our framework enhances LLM code repair via human-inspired mechanisms (multi-view interpretation, iterative refinement, experience storage, structure-guided historical reasoning), aiming for higher quality patches in fewer iterations to improve effectiveness and narrow the human-AI gap. Future work includes empirical validation (e.g., SVEN, PrimeVul), extension to broader bug types, adding dynamic analysis feedback, and optimizing reasoning/storage components for scale. Ultimately, we aim for AI coding assistants that learn and evolve like humans.

References

[1] Manish Acharya, Yifan Zhang, Kevin Leach, and Yu Huang. 2025. Optimizing code runtime performance through context-aware retrieval-augmented generation. *arXiv preprint arXiv:2501.16692* (2025).

[2] Yangruibo Ding, Yanjun Fu, Omniyyah Ibrahim, Chawin Sitawarin, Xinyun Chen, Basel Alomair, David Wagner, Baishakhi Ray, and Yizheng Chen. 2024. Vulnerability detection with code language models: How far are we? *arXiv preprint arXiv:2403.18624* (2024).

[3] Jingxuan He and Martin Vechev. 2023. Large language models for code: Security hardening and adversarial testing. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 1865–1879.

[4] Nan Jiang, Kevin Liu, Thibaud Lutellier, and Lin Tan. 2023. Impact of Code Language Models on Automated Program Repair. In *Proceedings of the 45th International Conference on Software Engineering (ICSE)*. IEEE/ACM, 1430–1442. doi:10.1109/ICSE48619.2023.00125

[5] Matthew Jin, Syed Shahriar, Michele Tufano, Xin Shi, Shuai Lu, Neel Sundaresan, and Alexey Svyatkovskiy. 2023. InferFix: End-to-End Program Repair with LLMs over Retrieval-Augmented Prompts. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, 1646–1656. doi:10.1145/3611643.3616240

[6] Zachary Karas, Aakash Bansal, Yifan Zhang, Toby Li, Collin McMillan, and Yu Huang. 2024. A tale of two comprehensions? analyzing student programmer attention during code summarization. *ACM Transactions on Software Engineering and Methodology* 33, 7 (2024), 1–37.

[7] Jiliang Li, Yifan Zhang, Yu Huang, and Kevin Leach. 2024. Malmixer: Few-shot malware classification with retrieval-augmented semi-supervised learning. *arXiv preprint arXiv:2409.13213* (2024).

[8] Theo Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. 2024. Is Self-Repair a Silver Bullet for Code Generation?. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.

[9] Boyang Yang, Haoye Tian, Weiguo Pian, Haoran Yu, Haitao Wang, Jacques Klein, Tegawendé F. Bissyandé, and Shunfu Jin. 2024. CREF: An LLM-Based Conversational Software Repair Framework for Programming Tutors. In *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*.

[10] Yifan Zhang, Chen Huang, Kevin Cao, Yueke Zhang, Scott Thomas Andersen, Huajie Shao, Kevin Leach, and Yu Huang. 2022. Pre-Training Representations of Binary Code Using Contrastive Learning. *arXiv preprint arXiv:2210.05102* (2022).

[11] Yifan Zhang, Chen Huang, Zachary Karas, Dung Thuy Nguyen, Kevin Leach, and Yu Huang. 2025. Enhancing Code LLM Training with Programmer Attention. *arXiv preprint arXiv:2503.14936* (2025).

[12] Yifan Zhang, Jiliang Li, Zachary Karas, Aakash Bansal, Toby Jia-Jun Li, Collin McMillan, Kevin Leach, and Yu Huang. 2024. Eyetrans: Merging human and machine attention for neural code summarization. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 115–136.